

Fitting Varieties

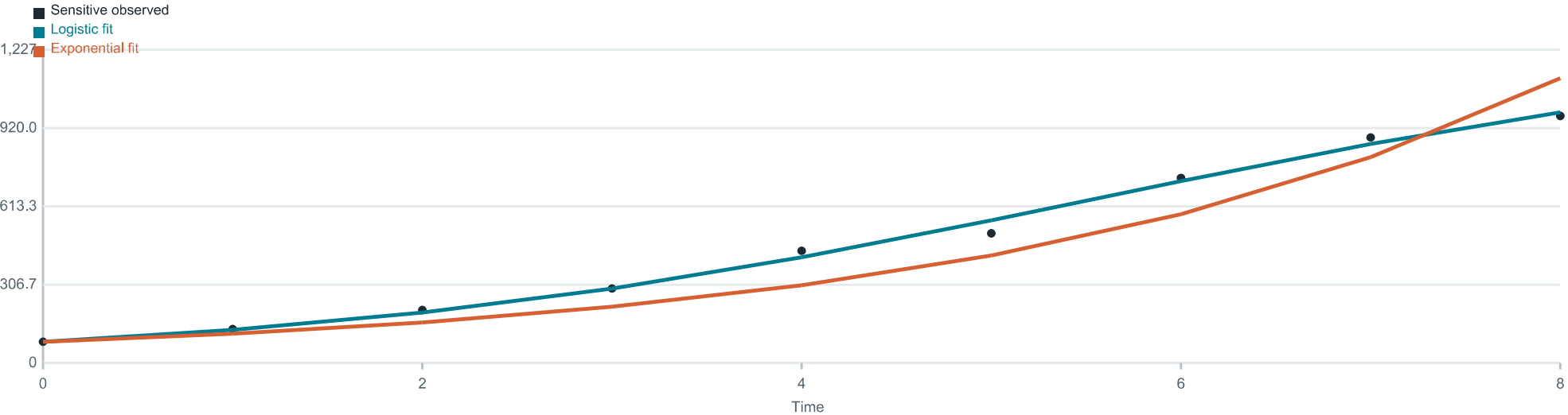
Visual examples of the core fitting APIs using the same seeded manufactured data as the function tour

Function	Question it answers	Example in this guide
compare_models	Which of two biological hypotheses better explains one time series?	Logistic versus exponential sensitive monoculture.
compare_datasets	How do parameter estimates differ across two separately fit conditions?	Sensitive versus resistant untreated monocultures.
fit_model	How do I fit a registered model specification to one condition?	Registry logistic model fit to sensitive monoculture.
fit_condition	How do I screen registered models across a labeled condition or dose group?	Sensitive drug-response series across three doses.
run_joint_fit	How do I fit one coupled model to multiple observed states?	Sensitive and resistant coculture states together.
compare_joint_models_dict	Which coupled mechanism has the best joint evidence?	Independent, symmetric, and asymmetric competition.

compare_models

Use this when one data set has two clearly stated candidate mechanisms. The function fits both models under their own starts and bounds, records BIC and SSR, writes a comparison CSV, and returns the lower-BIC model. Here the same noisy sensitive untreated data are tested against logistic and exponential growth.

Sensitive untreated monoculture



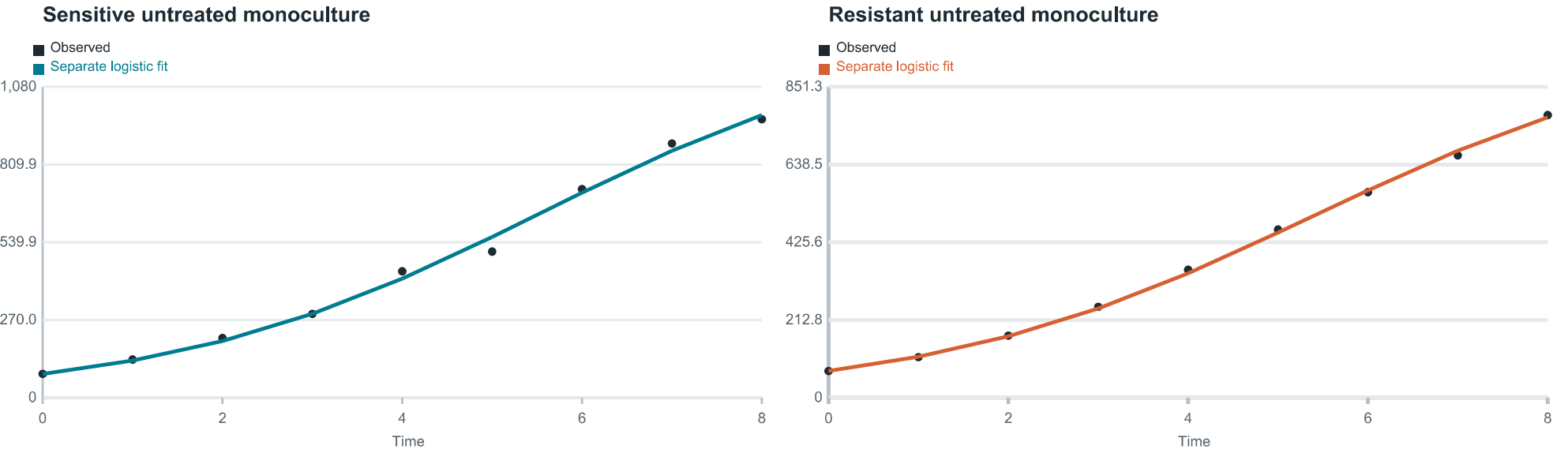
```
logistic_ode = Models.to_ode!(Models.build_logistic())
exponential_ode = Models.to_ode!(Models.build_exponential())
result = compare_models(times, sensitive_counts,
    "logistic", logistic_ode, [0.45, 1300.0],
    "exponential", exponential_ode, [0.35];
    bounds1=[(0.01, 1.2), (200.0, 2500.0)],
    bounds2=[(0.01, 1.2)], output_csv="model_comparison.csv")
```

Candidate	BIC	SSR	Interpretation
Logistic growth	60.038	4,358	Lower BIC is preferred; logistic matches the saturating manufactured trajectory.
Exponential growth	84.183	81,367	Useful null comparison when saturation is uncertain.

2. TWO-DATASET COMPARISON

compare_datasets

Use this when the model form is the same but you want condition-specific estimates. It is ideal for a control-versus-treatment, cell-line, or sensitive-versus-resistant comparison. It does not force parameter sharing; it returns one fit for each input data set and writes a compact CSV summary.



```
result = compare_datasets(times, sensitive_counts, "sensitive untreated",
logistic_ode, [0.45, 1300.0],
times, resistant_counts, "resistant untreated",
logistic_ode, [0.45, 900.0],
bounds1=bounds, bounds2=bounds, output_csv="dataset_comparison.csv")
```

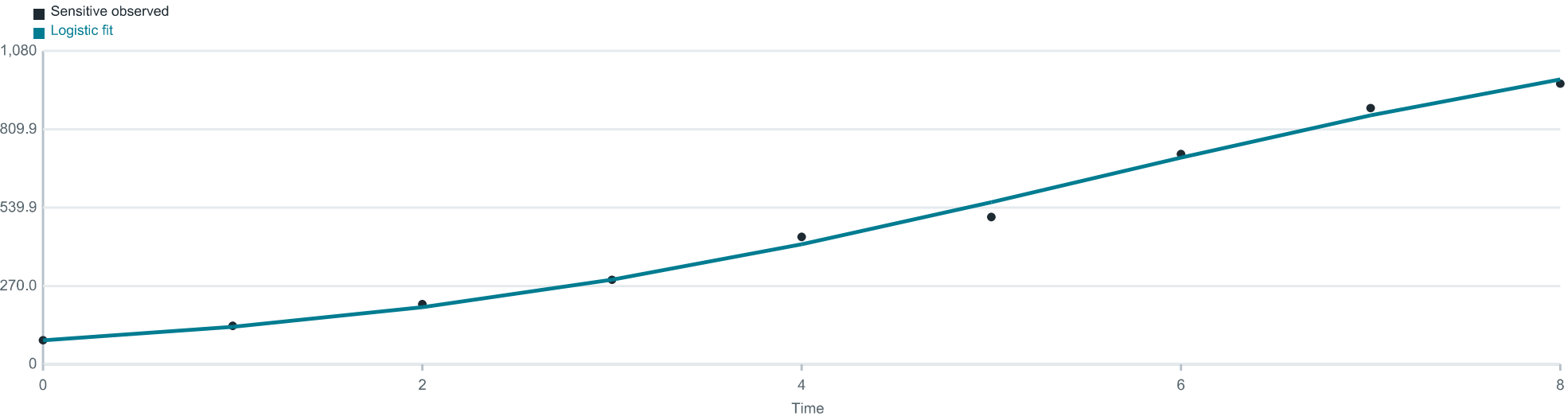
Dataset	Data-generating behavior	What separate fits reveal
Sensitive untreated	Logistic growth with seeded noise	Estimated $r=0.482$; $K=1,280$. Each condition receives its own parameter estimate.
Resistant untreated	Logistic growth with seeded noise	Estimated $r=0.470$; $K=991.4$. Each condition receives its own parameter estimate.

3. SINGLE REGISTERED-MODEL FIT

fit_model

Use this API when you have selected one registry model and need a fit with registry defaults, optional drug exposure, bounds, and fixed/anchored parameters. This is especially helpful in a reproducible pipeline because the ModelSpec carries the model metadata with the fitting call.

Sensitive untreated monoculture



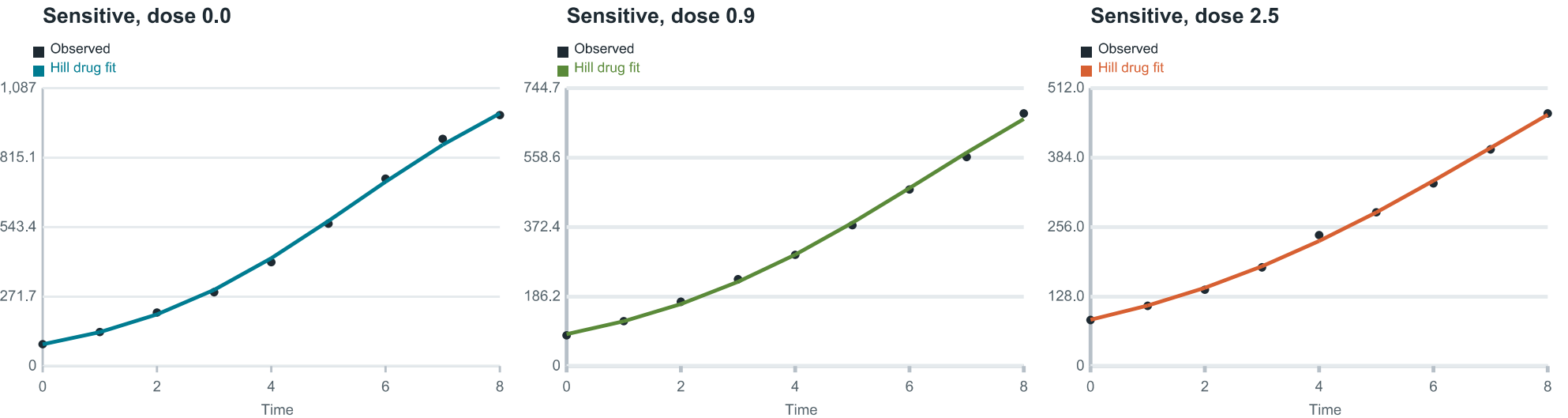
```
spec = get_model("logistic_growth")
result = fit_model(spec, times, sensitive_counts, 0.0;
p0=[0.45, 1300.0],
anchor_params=Dict{Int, Float64}(),
maxiters=5_000)
println((params=result.params, bic=result.bic, ssr=result.ssr))
```

Input	Role in the fit
ModelSpec	A registered model contains parameter names, bounds, solver, observable, and initial-guess information.
x and y	One ordered time series: the sensitive untreated manufactured counts.
dose / anchor_params	Optional exposure and fixed parameter values. Anchors let a later stage inherit trusted estimates.
Returned values	params, BIC, SSR, and solver status. This example's logistic reference fit has r=0.482, K=1,280.

4. CONDITION-LEVEL MODEL SCREEN

fit_condition

Use this for a labeled condition, optionally split into dose or replicate groups, when several registry ModelSpecs should be tried systematically. The example extends the untreated sensitive trajectory with the same drug-response data used in the function tour. Earlier untreated growth estimates can be passed as anchors so this stage focuses on drug-effect terms.



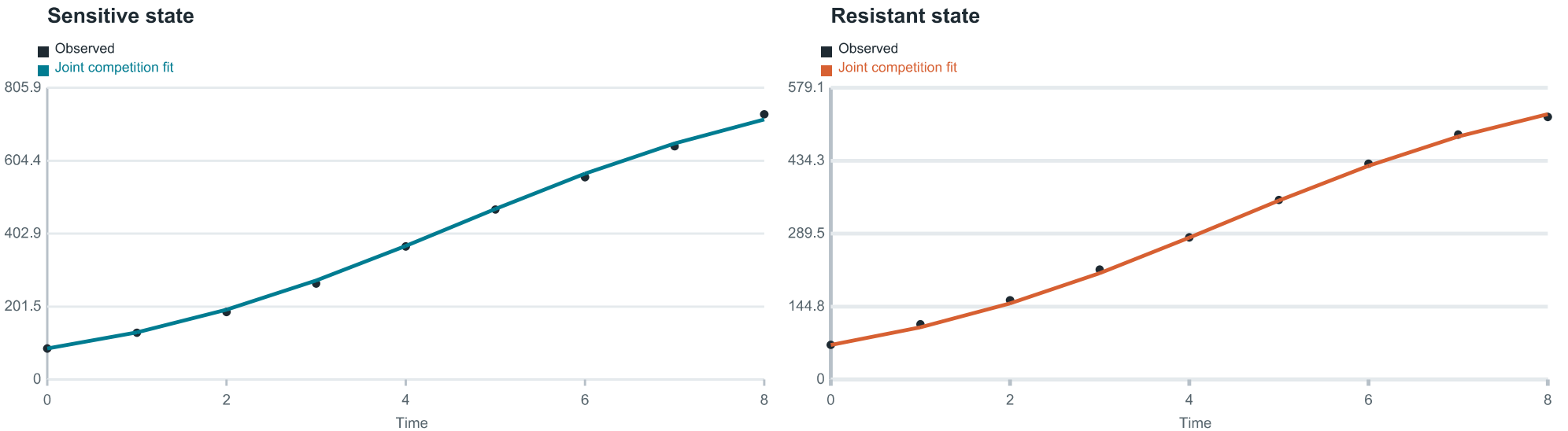
```
condition_rows = subset(data, :condition_name => ByRow(=="treated_sensitive"))
models = [get_model("logistic_growth"), get_model("hill_inhibition")]
summary = fit_condition(condition_rows, "treated_sensitive", models;
time_col=:time, measurement_col=:count, dose_col=:dose,
group_cols=[:dose], untreated_baseline=(r=0.48, K=1300.0))
```

Dose	Observed data	Fitted mechanism	Why this grouping matters
0.0	Sensitive monoculture with seeded noise	Logistic growth + Hill drug effect	The condition label and dose identify the exposure passed to each registered candidate.
0.9	Sensitive monoculture with seeded noise	Logistic growth + Hill drug effect	The condition label and dose identify the exposure passed to each registered candidate.
2.5	Sensitive monoculture with seeded noise	Logistic growth + Hill drug effect	The condition label and dose identify the exposure passed to each registered candidate.

5. JOINT MULTI-STATE FIT

run_joint_fit

Use this when several measurements arise from one coupled biological system and must be explained by one parameter vector. Unlike separate fits, this preserves the mathematical interaction between the sensitive and resistant states. The coculture fit below estimates asymmetric competition from both trajectories at once.



```
datasets = [(x=times, y=sensitive_coculture, state_index=1),
            (x=times, y=resistant_coculture, state_index=2)]
result = run_joint_fit(competition_odel, datasets, [85.0, 70.0],
                      [0.48, 0.50, 1300.0, 900.0, 0.6, 0.4];
                      bounds=[(0.01,1.2), (0.01,1.2), (200,2500), (200,2500), (0,1.5), (0,1.5)])
```

Dataset specification	Meaning	Why it helps
(x=times, y=sensitive_counts, state_index=1)	Observations map to state S in a coupled ODE.	The sensitive trajectory informs both its own growth and competition with R.
(x=times, y=resistant_counts, state_index=2)	Observations map to state R in the same coupled solution.	The resistant trajectory constrains the shared interaction terms at the same time.

6. JOINT MODEL SELECTION

compare_joint_models_dict

Use this immediately after defining multiple coupled hypotheses. It repeatedly calls `run_joint_fit`, ranks the joint BIC values, and writes a comparison table. This is the natural choice when the question is not merely whether parameters differ, but which multi-state mechanism best explains all observations together. In this example, the asymmetric competition model is selected because it has the lowest joint BIC.

Delta BIC (lower is better)



```
specs = Dict(  
  "independent" => (model=independent_ode!, p0=[0.48,0.50,1300.0,900.0]),  
  "symmetric" => (model=symmetric_competition!, p0=[0.48,0.50,1300.0,900.0,0.5]),  
  "asymmetric" => (model=competition_ode!, p0=[0.48,0.50,1300.0,900.0,0.6,0.4])  
)  
fits = compare_joint_models_dict(datasets, [85.0, 70.0], specs;  
  output_csv="joint_model_comparison.csv")
```

A compact decision guide

The fitting APIs differ mainly in what is shared and what is being compared. Start with the simplest function that matches the scientific question, then use joint fitting only when the biology genuinely couples the observations.

Situation	Use	What is shared?
One curve, two hypotheses	<code>compare_models</code>	Nothing: each candidate has its own parameters.
Same model, two conditions	<code>compare_datasets</code>	Model form only; fits remain separate.
One chosen registered model	<code>fit_model</code>	Registry metadata and optional anchored parameters.
One labeled condition with candidate registry models	<code>fit_condition</code>	Condition filters, group labels, and optional baseline anchors.
Several measured states from one ODE	<code>run_joint_fit</code>	One coupled solution and one parameter vector.
Several coupled hypotheses	<code>compare_joint_models_dict</code>	Joint data set; candidates are ranked by BIC.
More than two one-curve candidates	<code>compare_models_dict</code>	Same data, many independently fit candidate models.

Joint fitting gains a full uncertainty and identifiability layer

The existing fitting examples remain unchanged. These pages add the checks needed when a shared parameter vector is fit across sensitive, resistant, damaged, or macrophage-associated series.

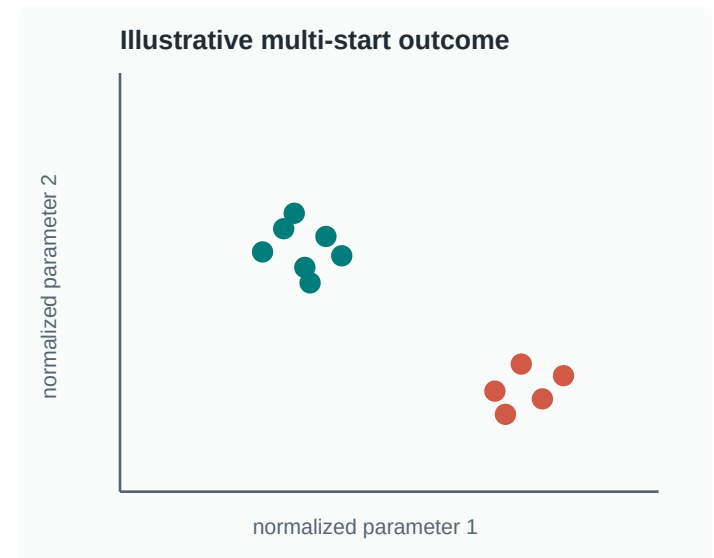
Function	Typed core inputs	Use correctly by
generate_multistarts	bounds::Vector{Tuple}; n_starts::Int	sample wide, reproducible starts; use :log for multi-order bounds
fisher_information	model::Function, dataset_specs, u0, p	supply residual_scale for every measured series
profile_likelihood	model::Function, dataset_specs, u0, p	refit nuisance parameters over scientifically defensible bounds
bootstrap_joint_fit	model::Function, dataset_specs, u0, p0	use residual or parametric resampling within each series

- The original bootstrap confidence intervals are retained. The new joint bootstrap reports 2.5% and 97.5% quantiles, plus failed-fit rate, for each parameter.
- A single dataset_specs entry must represent one measured series or replicate stratum, with x, y, state_index or observable, and residual_scale.

A reproducible numerical gate for high-dimensional ODE fits

Use this after `run_joint_fit` or `run_joint_multistart`, especially when prior growth, drug, and competition parameters are carried across staged conditions.

```
report = practical_identifiability_report(  
    four_state_model!, datasets, initial_state, initial_guess;  
    config=IdentifiabilityConfig(parameter_names, bounds;  
        n_starts=40, profile_points_per_side=12,  
        bootstrap_replicates=200, bootstrap_method=:parametric),  
    optimizer=:nelder_mead, maxiters=20_000)
```



- Review `report.fit`, `report.multistart.summary`, `report.solution_clusters`, `report.fisher`, `report.profiles`, and `report.bootstrap` together.
- `passes_numerical_gates` is a screening result, not a structural-identifiability claim.

9. STRUCTURAL ANALYSIS

Use the symbolic backend only after the observation process is explicit

Numerical fits use the ODE RHS plus data. Structural analysis repeats the ODE with symbolic outputs and states exactly what the assays observe.

Question	Function	Output
Do mapped measurements match the joint data?	<code>validate_observation_map</code>	direct state or custom observable record
Which parameters are globally, locally, or not identifiable?	<code>structural_identifiability</code>	symbolic classification table
Is a symbolic backend still required?	<code>structural_identifiability_report</code>	explicit requirement, never a false proof
Could the planned experiment recover truth?	<code>synthetic_recovery_benchmark</code>	bias, RMSE, and recovery success rate

- For macrophage or multiple-damage-compartment models, design experiments that independently observe affected populations, perturb known inputs, and contain enough time points to distinguish transitions.
- Report bootstrap 95% intervals, profile bounds, multi-start clusters, and symbolic classification together in the final biological analysis.

Use PRCC to screen the entire allowed range of every parameter

This function is a global complement to local Fisher sensitivity. It samples the biological bounds rather than perturbing one parameter only around a fitted estimate.

```
global = global_sensitivity_analysis(  
    model::Function, dataset_specs, u0;  
    bounds::Vector{Tuple}, parameter_names::Vector{Symbol},  
    n_samples::Int=400, scale=:log)  
  
global.summary  
global.pointwise
```

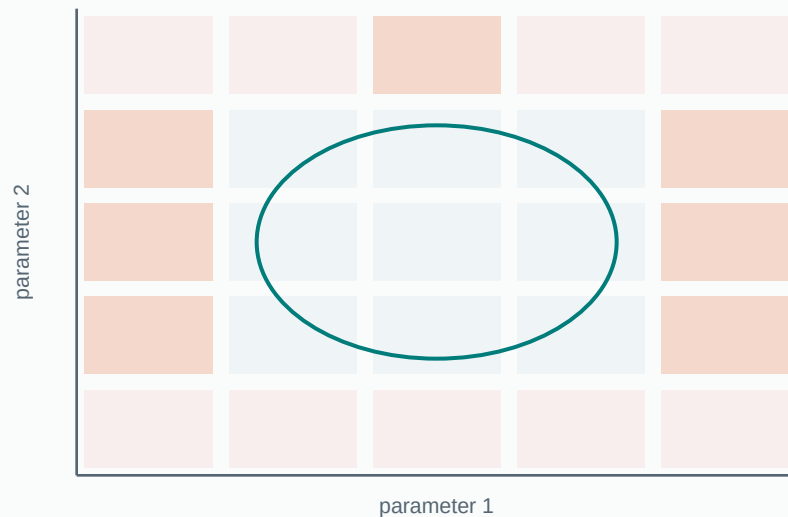
- The returned sample audit records successful, invalid, and failed simulations.
- summary ranks parameters by mean and maximum absolute PRCC across all observed outputs.
- pointwise PRCC identifies when a parameter matters most and which measured population is most informative.

11. PAIRED PROFILE LIKELIHOODS

Profile the parameter pair that biology says may be confounded

For example, drug damage and macrophage killing can both reduce viable cells. A paired profile tests whether the observations actually separate them.

Illustrative paired-profile surface



```
paired = paired_profile_likelihood(  
    model, dataset_specs, u0, p0;  
    bounds=bounds, parameter_names=parameter_names,  
    pair=( :drug_damage, :macrophage_killing),  
    points_per_side=10)  
paired.region
```

- All nuisance parameters are re-fit at every grid point.
- `accepted_region` is the joint 95% region; a `bound_touching` status signals unresolved joint uncertainty.

Pool every group while allowing specific biological effects to differ

Use this when the experiment contains related conditions such as sensitive and resistant strains, doses, or macrophage conditions that should inform one another.

Input	Type	Use
groups	Vector of named groups	each group has name, dataset_specs, and u0
varying_parameters	Vector{Symbol}	only these get group-specific log-scale effects
random_effect_sd	Real or Vector{Real}	fixed partial-pooling strength
output	group_parameters	central, group-specific, and log-deviation estimates

- All observations are fit together. Smaller groups borrow strength from the shared central parameter estimate.
- The output includes pooled BIC, group-wise residual contributions, and predictions for every group.
- This is a penalized hierarchical likelihood. It is not yet a full Bayesian posterior with estimated population variances.